

Web Platform Analysis

Date: June 16, 2026

Discipline: Offensive and Defensive Cybersecurity

Lecturer: Carlos Antunes

Team: João Marto

Table of Contents:

1.Introduction and Framework.....	3
1.1.Laboratory Objectives	3
1.2.Scope and Scenario of Tests	3
2.Web Platform Recognition (Fingerprinting)	4
3.Fuzzing and Directory Mapping.....	5
4.Active Web Vulnerability Analysis.....	7
5.Conclusions	8
6.Command Log	9

1. Introduction and Framework

This report documents the execution of **Laboratory 8 – Web Platform Analysis**, integrated in the curricular unit of Offensive and Defensive Cybersecurity. This phase focuses exclusively on the application layer (Layer 7 of the OSI model), aiming at the identification, mapping and exploitation of vulnerabilities in Web services and platforms.

1.1. Laboratory Objectives

In accordance with Practice Sheet 8, the tasks set out consist of:

- Perform configuration tests and collection of information (*Fingerprinting*) on the target web server.
- Compare the physical structure with the indexable structure of the application, using directory *fuzzing* techniques .
- Test potential web platform programming and configuration vulnerabilities.
- Prepare this technical report based on the evidence collected.

1.2. Scope and Scenario of Tests

To carry out these tests in a controlled and authorized environment, the **Local Router Management Web Platform** (AltiCe/MEO Gateway Gateway), accessible through the internal IP address [IP Redacted] in the standard HTTP/HTTPS port, was defined as a target. This target presents a realistic scenario of auditing *IoT hardware*/Infrastructure and the *web servers* embedded in these equipment.

2. Web Platform Recognition (Fingerprinting)

The first phase of the application analysis focused on the passive collection of *banners* and HTTP metadata from the embedded web server on the *local router* ([IP Redacted]). The goal is to identify the underlying technologies without performing intrusive scans.

For this purpose, the *whatweb* utility was used, as recommended in the enumeration methodologies.

Command used:



Fingerprinting: The *output* returned by *whatweb* reveals crucial architectural details about the equipment. The web service is supported by **micro_httpd**, an extremely small and lightweight HTTP server, classically used in embedded systems and IoT equipment due to its low resource consumption. The Title[FGW] tag confirms that it is the **FiberGateway** administration panel (Altice/MEO). A robust defensive posture on the part of the manufacturer stands out: the root page immediately returns a 401 Unauthorized error, indicating that there is no public content accessible without prior authentication. Additionally, the presence of essential security headers was verified, such as X-Frame-Options[deny] (prevention of *Clickjacking* attacks) and X-XSS-Protection[1; mode=block] (reflected *Cross-Site Scripting* mitigation).

3. Fuzzing and Directory Mapping

Once the basic technology was understood, the visible structure was compared with the indexable structure of the platform, in order to discover hidden directories, configuration files or secondary administration panels. The dirb tool associated with the standard dictionary (common.txt) was used.

Command used:

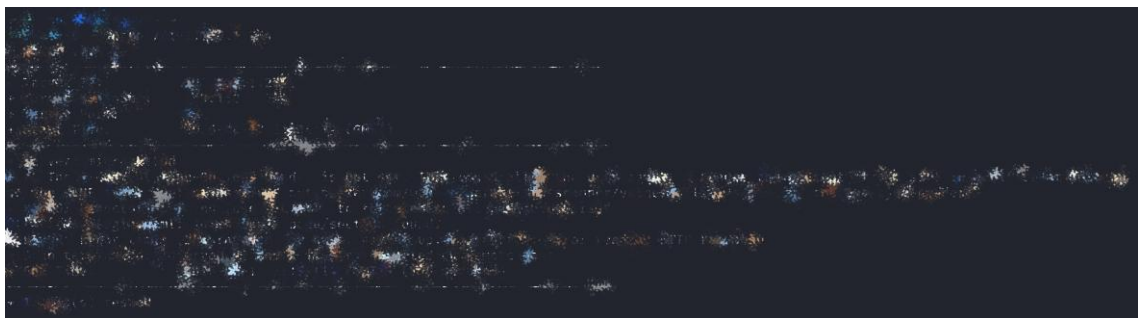


Analysis Note (Directory Mapping): The attack of *Fuzzing* Over the course of 4612 words, it generated expected results for an embedded server. The dirb has located 10 paths on the server. The overwhelming majority of discovered files (such as admin.cgi, cachemgr.cgi, and cgi-bin/) returned the 401 Unauthorized code. This indicates that the web architecture of the *Router* is based on CGI scripts (*Common Gateway Interface*) for the execution of administration tasks, but access control is implemented globally, not allowing the *bypass* straight to the scripts. The only freely accessible file (Code 200 OK) was swfobject.js, a static resource.

4. Active Web Vulnerability Analysis

For the active exploitation phase of vulnerabilities and *misconfigurations*, the *nikto* web vulnerability scanner was applied. Given the aggressive nature of this tool, we sought to validate the robustness of the *micro_httpd* server in the face of anomalous requests.

Command used:



Analysis Note (Active Scan): The execution of Nikto revealed very interesting behaviors about the resilience of the equipment. In terms of structural vulnerabilities, the *scanner* detected the absence of the X-Content-Type-Options header, which would theoretically allow *MIME-sniffing* attacks (although the impact is negligible on a closed home router). It has also successfully mapped the direct administrative entry point in `/login.html`. However, the main conclusion is drawn from the forced interruption of the process: **ERROR: Error limit (20) reached for host**. This breakdown in communication is due to the fact that IoT equipment has very limited computational resources. By being the target of thousands of simultaneous and aggressive HTTP requests by nikto, the server *has micro_httpd* exhausted its processing capacity or activated *rate-limiting* mechanisms, actively dropping network packets and stopping the attacker's scanning.

5. Conclusions

Lab 8 allowed the successful application of Web audit methodologies (OSINT, Fuzzing and Scanning) in a real scenario of home infrastructure. The following conclusions are drawn:

Attack Surface Reduction: FiberGateway's web deployment model follows good security practices, globally blocking any unauthenticated access (HTTP 401) and utilizing native security headers against XSS and Clickjacking.

Resilience to Automated Tools: Practical testing with nikto has shown that the use of noisy *scanners* against embedded infrastructures (*routers, switches, IoT*) often results in *Denial of Service* or *Rate-limiting*). In critical environments, an approach purely based on these automated tools would not only be detected immediately, but could temporarily bring down the legitimate service.

6. Command Log

Web Information Collection (Fingerprinting):

- whatweb http:// [IP Redacted]

Directory Fuzzing:

- dirb [http:// \[IP Redacted\]](http:// [IP Redacted])

Vulnerability Scanning:

- nikto -h http:// [IP Redacted]